Before Development Checklist

How to use this checklist

No developer writes code, or opens an AI coding tool, until this checklist is signed off by the tech lead. One hour of planning here saves weeks of rework later. If an item does not apply, write "N/A" and the reason; never skip it silently.

It is built from how the biggest engineering companies start projects:

Company	Practice	What we borrow
Amazon	"Working backwards" docs, one-way vs two-way door decisions	Write the problem first; slow down only on decisions you cannot undo
Google	Design docs reviewed before coding	Every feature has a short design doc
AWS	Well-Architected Framework (6 pillars)	Check security, reliability, performance, cost, operations, sustainability
Microsoft	Security Development Lifecycle (SDL), threat modeling	Security planned before code
OWASP	ASVS and Top 10	Security requirements list
Heroku	12-factor app	Config, environments and deployability rules

The golden rule: **if it is not written down and reviewed, it is not planned.**

1. Requirements and scope

Everyone must agree on what "done" means before work starts.

- [ ] Problem statement in 3–5 lines: who the user is, what pain we solve, how we measure success
- [ ] User stories with acceptance criteria for every feature
- [ ] User roles listed (customer, admin, staff, super admin) and what each can do
- [ ] Non-functional requirements written down:
  - [ ] Expected users at launch and in 1 year
  - [ ] Peak requests per second
  - [ ] Response time target (e.g. p95 under 300 ms)

  - [ ] Uptime target (e.g. 99.9%)
  - [ ] Supported devices, browsers, OS versions, languages
- [ ] Out-of-scope list, so nobody builds extras
- [ ] Edge cases listed: empty states, slow or no network, double clicks, large files, timezones, currencies, special characters
- [ ] Third-party dependencies identified (payment, SMS, email, maps, AI APIs) with their limits and costs
- [ ] Legal needs identified early: personal data, payments, AI features, users in EU/UK, content posted by users
- [ ] UI/UX designs or wireframes approved, including error and loading states
- [ ] Timeline with milestones and buffer for testing (at least 20–30% of the plan)
- [ ] Stakeholder or client sign-off on scope

2. Architecture and design review

A short design doc, reviewed by another senior, before any code.

- [ ] Design doc (1–3 pages): goal, components, data flow, APIs, third-party services, risks
- [ ] Architecture diagram the whole team can explain
- [ ] Design reviewed by at least one other senior engineer
- [ ] Architecture Decision Records (ADR): what we chose, what we rejected, why
- [ ] Tech stack justified (not chosen just because it is trendy)
- [ ] Monolith vs microservices decided honestly; start simple unless scale demands otherwise
- [ ] Stateless services so they can scale horizontally; sessions in Redis or tokens, not memory
- [ ] Sync vs async decided: long tasks go to queues (RabbitMQ), not HTTP requests
- [ ] Failure plan for every dependency: database down, queue down, third-party API slow or down
- [ ] Retries with backoff, timeouts and circuit breakers planned
- [ ] Idempotency for payments, webhooks and queue consumers (the same message twice must not charge twice)
- [ ] Caching strategy: what to cache, for how long, how to invalidate
- [ ] Scalability plan: what breaks first at 10x users
- [ ] Multi-tenancy model (if B2B): how one customer's data is kept apart from another's
- [ ] Cloud cost estimate at launch and at 10x scale
- [ ] Observability planned from the start: logs, metrics, traces, request IDs

3. API and database design

APIs and schemas are hard to change after launch, so agree on them first.

API

- [ ] API contract written first (OpenAPI / Swagger) and shared with frontend and mobile teams
- [ ] Versioned from day one (/api/v1/)
- [ ] Consistent naming, HTTP methods and status codes
- [ ] Standard error format (code, message, request ID) for every endpoint
- [ ] Pagination, filtering and sorting on every list endpoint
- [ ] Rate limits planned per user and per IP
- [ ] Request and response validation (schema-based, e.g. Joi, Zod, Pydantic)
- [ ] Webhooks signed and verified
- [ ] Mobile apps: plan for old app versions still calling the API (force-update strategy)

Database

- [ ] Right database chosen for the data (relational vs document vs cache)
- [ ] Schema reviewed: primary keys, foreign keys, unique constraints, NOT NULL rules
- [ ] Indexes planned for every common query
- [ ] created_at, updated_at (and created_by where useful) on every table
- [ ] All times stored in UTC
- [ ] Money stored as integers or decimals, never floats
- [ ] Soft delete vs hard delete decided per table
- [ ] Migration tool chosen (Prisma, Knex, Alembic, Flyway); no manual changes to production schema
- [ ] Backup and restore plan agreed (RTO and RPO numbers)
- [ ] Expected data size in 1 year estimated

4. Security by design

Security added at the end is expensive and leaky; plan it now.

- [ ] Threat model done with STRIDE: what can be attacked, by whom, and the damage
- [ ] Authentication chosen: OAuth2 / OIDC or a proven provider (Azure AD B2C, Auth0, Cognito); never roll your own crypto
- [ ] Password rules: bcrypt or argon2 hashing, never plain text or MD5

- [ ] Token plan: short-lived access tokens, refresh tokens, logout and revoke
- [ ] MFA for admins (and users if sensitive data)
- [ ] Authorization model: roles, permissions, who can see whose data
- [ ] Every endpoint checks that the user owns the resource (prevents IDOR)
- [ ] Secrets plan: Key Vault or K8s secrets; never in code, never in committed .env files
- [ ] Encryption in transit (TLS 1.2+) and at rest (database, storage, backups)
- [ ] File uploads: type check, size limit, virus scan, private storage with signed URLs
- [ ] Admin panel separated and protected (VPN, IP allowlist or SSO + MFA)
- [ ] Audit log planned for sensitive actions (login, role change, delete, payment, export)
- [ ] Payments through a certified gateway (Razorpay, Stripe); never store card numbers
- [ ] Network plan: databases and queues not exposed to the internet; private endpoints

5. Data and privacy by design

India's DPDP Act is fully enforced from 13 May 2027, and GDPR already applies to EU and UK users. Building privacy in now is far cheaper than retrofitting it.

- [ ] Data inventory: every personal field we collect, why, where it is stored, who can access it
- [ ] Collect the minimum; if a field is not needed, do not store it
- [ ] Consent flow designed: clear notice, no pre-ticked boxes, consent stored with timestamp
- [ ] Users can withdraw consent as easily as they gave it
- [ ] Delete-my-data and export-my-data flows designed from day one, including backups and vendors
- [ ] Retention rule for every data type, with automatic deletion
- [ ] Data location decided (India region for Indian users where required; RBI rules for payment data)
- [ ] Sensitive fields encrypted or masked (phone, email, ID numbers)
- [ ] PII never written to logs (no passwords, tokens, OTPs, card numbers, phone numbers)
- [ ] Analytics and tracking only after consent where the law requires it
- [ ] Vendors that touch personal data listed; DPAs planned
- [ ] Children's data: verifiable parental consent if users can be under 18
- [ ] Test data is fake; never copy real production data to dev

6. Coding standards and AI / vibe coding rules

Coding standards

- [ ] Style guide and folder structure agreed per language (Node.js, Python, React)
- [ ] Linter and formatter configured (ESLint, Prettier, Ruff, Black) and enforced in CI
- [ ] TypeScript strict mode or Python type hints
- [ ] Error-handling pattern agreed: no empty catch blocks; users see friendly messages, logs get details
- [ ] Logging standard: JSON logs, log levels, request ID on every line, no PII
- [ ] Timeouts on every external call by default
- [ ] Testing standard: what needs unit, integration and end-to-end tests; coverage target (e.g. 70%+ on core logic)
- [ ] PR rules: small PRs (under ~400 lines), clear description, screenshots for UI, linked ticket
- [ ] Definition of Done: code reviewed, tests pass, docs updated, deployed to staging

AI-assisted and vibe coding rules

AI writes code fast, but the developer who merges it owns it. Treat AI output like code from a new intern: useful, always reviewed.

- [ ] Use only company-approved AI tools and accounts
- [ ] Never paste secrets, customer data, private keys or confidential client code into AI tools
- [ ] Every AI-generated line goes through normal PR review; the author must be able to explain it
- [ ] Verify every package the AI suggests actually exists, is maintained and is popular (fake package names are a real attack)
- [ ] No AI-written auth, payment, encryption or permission logic merged without senior review
- [ ] AI code must come with tests, and those tests must fail when the code is broken
- [ ] Check for common AI mistakes: hard-coded values, missing error handling, SQL built from strings, disabled SSL checks, outdated APIs, invented functions
- [ ] Give AI tools project context (coding standards file, architecture notes) so output matches our patterns
- [ ] Check licences: do not paste large copied code of unknown origin

If the product itself uses AI

- [ ] Users are told when they are talking to AI; AI-generated content is labelled
- [ ] Protect against prompt injection: never let model output run commands or queries unchecked
- [ ] Human approval for high-impact AI actions (payments, emails, deletes)
- [ ] Prompts and outputs logged without PII, for debugging and audits
- [ ] Cost limits and rate limits on AI API usage
- [ ] Evaluation set to test AI quality before every model or prompt change

7. Repository, environments and CI/CD setup

Set up the safety rails on day one, while the codebase is still empty.

Repository

- [ ] Branch protection on main: no direct push, PR + at least 1 approval + green CI
- [ ] Branching strategy agreed (trunk-based or GitFlow) and branch naming convention
- [ ] Commit message convention (e.g. Conventional Commits)
- [ ] .gitignore covers .env, keys, build output, node_modules
- [ ] CODEOWNERS file so the right person reviews sensitive areas
- [ ] README: how to run locally in under 15 minutes
- [ ] PR template with a checklist (tests, docs, security, screenshots)

CI pipeline (runs on every PR)

- [ ] Lint, format and type checks
- [ ] Unit and integration tests
- [ ] Secret scanning (GitHub secret scanning, Gitleaks)
- [ ] Dependency scanning (Dependabot, Snyk) and container image scanning (Trivy)
- [ ] Static code analysis (SonarQube or CodeQL)
- [ ] Build fails on critical issues; no manual override without a lead

Environments

- [ ] Separate dev, staging and production, with separate databases, keys and cloud accounts or resource groups
- [ ] Staging mirrors production (same versions, same config shape)
- [ ] Infrastructure as code (Terraform, Bicep, Helm); no clicking in the portal

- [ ] Config through environment variables (12-factor); nothing environment-specific in code
- [ ] Docker images built once and promoted dev → staging → production
- [ ] Least-privilege access: developers do not have write access to production by default
- [ ] Cloud budget alerts set from the first day

Common planning-stage mistakes

Mistake	What happens later	Fix now
Starting to code from a chat message, no written scope	Endless changes, arguments over "done"	Written acceptance criteria
No API contract	Frontend and backend build different things	OpenAPI spec first
Storing money as float	Rounding errors in invoices	Integers or decimals
Storing local time	Wrong dates across timezones	Store UTC
No delete-account design	Painful rework for DPDP / GDPR	Design it in section 5
Secrets in .env pushed to git	Leaked keys within minutes	.gitignore + secret scanning
One shared database for dev and prod	Test data reaches real customers	Separate environments
Trusting AI code without reading it	Hidden security holes	Review rules in section 6
No testing time in the plan	Bugs found by customers	20–30% buffer for testing
Picking new tech nobody knows	Slow delivery, fragile code	Justify in an ADR

Sign-off before coding starts

Area	Owner	Signed
1. Requirements and scope	Product owner + tech lead	

Area	Owner	Signed
2. Architecture and design review	Tech lead + reviewing senior	
3. API and database design	Backend lead	
4. Security by design	Tech lead	
5. Data and privacy by design	Tech lead + product / legal	
6. Coding standards and AI rules	Whole team (read and agreed)	
7. Repository, environments and CI/CD	DevOps owner	

Next step after coding: the Before Launch Checklist.